

Wind Tunnels & Bonsai Servers

Arun Darlie Koshy – Sr Engineer, HPE

Jean-Marie Verdun – Sr Distinguished Technologist, HPE

OCP PLATINUM® Member

Agenda

- **Where we were – Wind Tunnels for ASI (OCP 2025 recap)**
- **Wind Tunnels: progress, OCPTV integration & what changed**
- **Wind Tunnels: next steps and call to action**
- **The bridge – from testing models to streaming firmware**
- **Firmware streaming PoC: Ethernet, TinyGo, SPI-NOR**
- **Bonsai servers – tiny, OS-less, streamed at boot**
- **How it all connects**

Recap – Wind Tunnels for ASI

- **A private AI / ML cluster giving real-world testing for frontier models & ML/AI techniques**
- **Key themes we set out:**
  - We are far from AGI (& arguably ASI)
  - Tests are key for production AI/ML models
  - Private / personal models may win the long haul
- **Cracks in the foundation models:**
  - Hallucinations, jailbreaks & indirect prompt injection have no 'fix'
  - Power/compute consumption signal (human brain ~ 20 watts)
  - Scaling keeps paying off – curve-fitting vs. causality
- **Jumpstart: clone → set up on preferred gear → Hello World**
  - *github.com/opencomputeproject/ocp-diag-windtunnel*

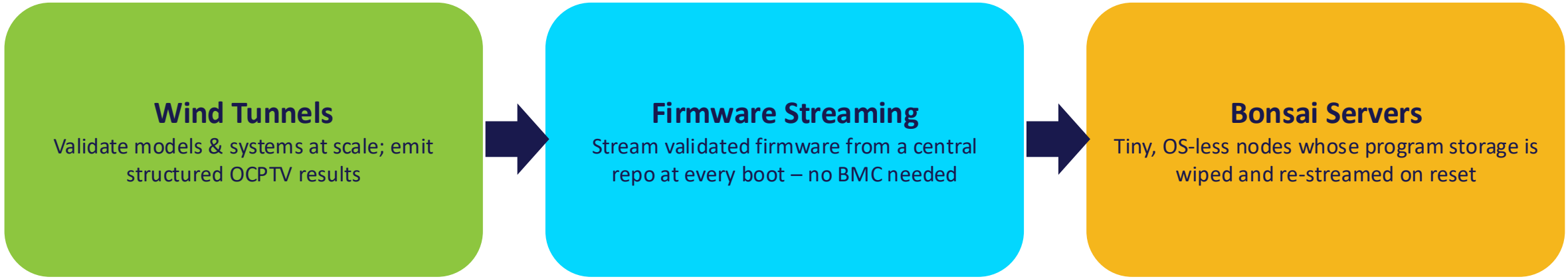
Wind Tunnels – Progress Since Last Summit

- **Repo live & maintained: ocp-diag-windtunnel (Shell + Python)**
  - Sets up a private AI/ML cluster; initial T&V focus, expanding to other OCP areas
- **OCPTV output integration demonstrated on the cluster**
  - *import ocptv.output as tv – TestRun / Step / Log / DiagnosisType*
  - Structured, machine-readable results from live model runs (vLLM on HPE gear)
- **Interoperates with the broader OCP diagnostics ecosystem**
  - ocp-diag-core · OSF-OSFCI
- **Framing tests for intelligence, not just correctness**
  - LLMs broke the Turing test · ARC-AGI · Millennium Problems (updates & controversy)

Wind Tunnels – Next Steps

- **Contribute frameworks, tests and agents for the OCP Wind Tunnel**
- **Develop a highly specialized set of tests for ML/AI agent contributions**
- **Private agents tailored to author contributions and run tests**
- **Broaden beyond the T&V context / scope**
- **Pool resources together for the community**
- Building & testing private AI

The Bridge: From Testing Models to Streaming Firmware



Same principle end to end: nothing trusted is kept locally – it is validated centrally, streamed on demand, and verified before it runs.

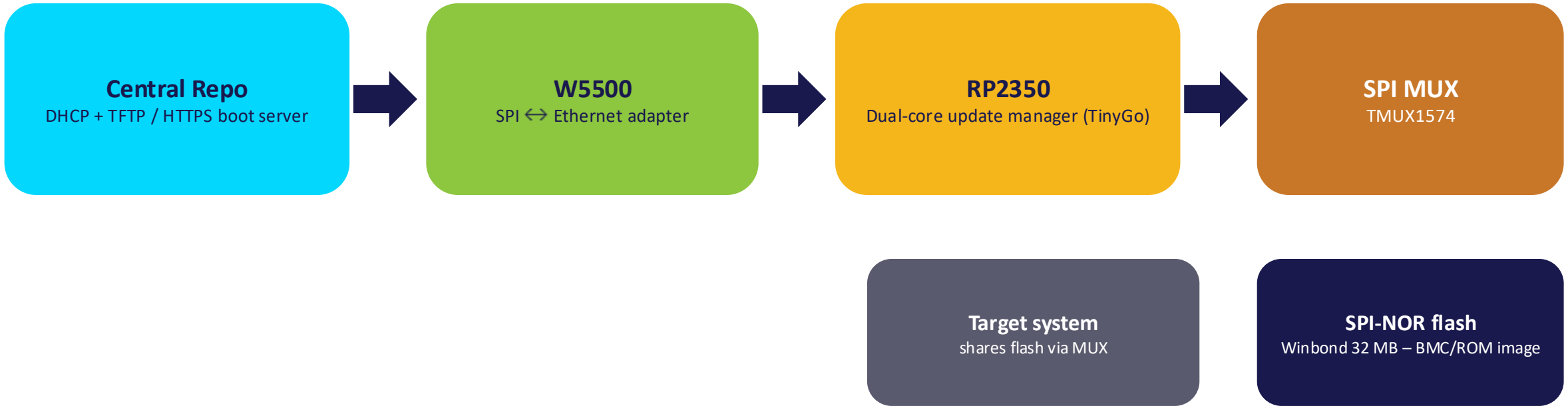
Firmware Update Today – Why It's Hard

- **BMC updates over I2C / USB / PCIe**
  - Each bus: own protocol, speed, electricals, reliability
  - Integrity: I2C is noise-prone → retransmit / recovery
  - Tight timing, power loss, safe rollback to avoid bricking
  - Shared lines → mux & arbitration
  - Per-path authentication & image validation
- **Historically avoided Ethernet for updates**
  - BOM: MAC + PHY + magnetics + RJ45
  - Needs a higher-end MCU (CPU, RAM, flash, MAC)
  - PCB layout: controlled impedance, isolation
  - More board area, power and dev/test cost
  - So designers chose I2C / SPI / UART / USB instead

The Approach – Ethernet + a Central Repo

- **One standardized interface: Ethernet / TCP-IP**
  - Built-in ordering, retransmission, error detection; higher bandwidth & reach
  - Mature security: TLS encryption + mutual auth + signature verification
- **Keep the low-cost MCU: SPI-to-Ethernet adapter (W5500-class)**
  - Centralizes MAC/PHY (and offloads) on a cheap dedicated chip → lower BOM
- **Write it in Go / TinyGo, not hand-rolled C**
  - Bounds-checked memory + GC kill whole classes of buffer / use-after-free bugs
  - Goroutines/channels for safe concurrency; mature TCP/TLS libraries
- **Endpoints poll a central repository at each boot**
  - Auto-check validated updates → atomic apply with rollback → single-point fleet mgmt

Firmware Streaming PoC – Architecture



Boot flow: DHCP → ARP → TFTP read · compare vs. SPI-NOR · update on mismatch · DeepSleep · hand flash back to target via MUX

Firmware Streaming PoC – Where It Stands

- **Working today**
  - TinyGo + TFTP client on RP2350 over W5500
  - DHCP → ARP → TFTP transfer, dual-core split
  - Tx/Rx queues + goroutines; GPIO21 IRQ driven
  - GC pressure monitored during transfer
- **Measured**
  - Firmware image 344 kB – fits in RP2350 SRAM
  - 4 MB in ~3.14 s ≈ 1.27 MB/s (peak ~1.6 MB/s)
  - 1400-byte TFTP blocks stay within MTU
- **Design choices**
  - Immutable boot ROM speaks only DHCP/ARP/TFTP
  - Complex TCP/HTTPS deferred to a 2nd-stage image in SRAM
  - Smaller attack surface in the trusted core
- **Why it matters**
  - Validate & update SPI-NOR (BMC/ROM) before boot
  - No human intervention; low-cost microcontroller
  - Fleet-wide consistency from one repository

Firmware Streaming – Roadmap

- **Concurrent SPI-NOR update during the TFTP transfer (in progress)**
- **Barcelona demo: flip-flop a full BMC firmware (~32 MB) live**
- **Two-stage boot: reset into an SRAM-executed secondary stage that fetches over HTTPS**
  - Second-stage IP stack pulled via TFTP into a PSRAM extension – keeps boot ROM minimal
- **Move off the W5500 Tx to single-pair Ethernet (T1, 2 wires)**
  - Automotive-grade switches → well suited to internal system management
- **Expose the SPI-Ethernet behind the MUX to the target**
  - Publish sensors & hardware errors over Ethernet instead of aging protocols
- **Beyond SPI-NOR: NAND & I2C EEPROM (RP2350 PIO / FPGA-like paths)**
- **Endgame: firmware update & management with no integrated BMC**
  - Build supercomputers with dramatically eased management

Enter Bonsai Servers – Small by Design

- **AI is going everywhere – but standard servers aren't AI-ready**
  - Consumer industry shifts fast; enterprise hardware adoption is slow
- **Shrink the server instead of scaling it up**
  - High-core CPUs pay coordination overhead: coherency, NUMA, locks, TLB conflict
  - …and a memory bottleneck plus OS complexity
  - Low-core CPUs waste less energy – no 'always-on' tax
- **Bonsai = a complete, deliberately tiny server node**
  - One node architecture for AI and general infrastructure
  - Optimized for power consumption and processing speed
  - No OS – bare-metal only, seamless management, derived from the embedded-world SDK

Bonsai Node – Architecture & Node Profiles

Compute Node

Scheduler Node

Management Node

Node profiles

Hardware envelope
2–8 cores · big cache · high frequency
4 GB / core · optional NAND · NPU
Mgmt µC + network 1 / 2.5 / 10 Gbps, low latency
Langs: Go · Rust · C / C++ · MicroPython

Program storage = streamed firmware
Code executes in place from program storage
Stack size is configurable
Storage is wiped on crash, reset or power-on
→ re-streamed & verified from the central repo

Bonsai – Value, Market & Positioning

- **Our value add**
  - Management at fleet scale
  - Software distribution (stream & verify)
  - ISA agnostic – RISC-V, ARM, x86-64
  - Simplicity as a feature
- **Positioning**
  - One solution won't fit all – but helps
  - Premium, non-managed, developer-oriented
  - Create a new market
- **Use cases & TAM**
  - 500M+ websites, ongoing AI integration
  - Simple hardware requirements
  - Useful to early adopters and startups
- **Where firmware streaming fits**
  - No integrated BMC needed for management
  - Every node connects to a management network (T1)
  - Validated images pushed from a single point

Bonsai – Proof of Concept

- **Software approach – a new 'ROM package'**
  - Hardware init via stripped UEFI or U-Boot
  - Minimalist Linux kernel with KVM support
  - Nano-hypervisor: AWS Firecracker
  - A single microVM running bare-metal with virtio – the user app
  - Runs on RISC-V, ARM and x86-64 bare metal (quad-core, 16 GB)
- **Hardware development**
  - Designed with AI assistance; ~$2000 budget
  - Targeting a sub-$5 solution – modern, efficient, secure

How It All Connects

- **Wind Tunnels give us the T&V harness**
  - Validate models, firmware and whole systems; emit structured OCPTV results
- **Firmware streaming is the delivery mechanism**
  - Pull validated, signed images from a central repo over Ethernet at boot
  - Trusted core stays tiny; TCP/HTTPS in a re-streamable second stage
- **Bonsai servers are the endpoint**
  - OS-less nodes; program storage wiped on reset and re-streamed on demand
  - No BMC required – managed over a single-pair Ethernet fabric
- **One loop: validate centrally → stream on demand → verify before run → test again**

Call to Action

Wind Tunnels: contribute frameworks, tests & agents; pool community resources

Firmware streaming: help move from TFTP to secure HTTPS + signature verification

Push single-pair (T1) Ethernet for internal management fabrics

Prototype Bonsai node profiles on your ISA of choice

Join us for at the innovation village

ocp-diag-windtunnel · ocp-fwupdater-PoC



References & Resources

- github.com/opencomputeproject/ocp-diag-windtunnel
- github.com/opencomputeproject/ocp-fwupdater-PoC
- github.com/opencomputeproject/ocp-diag-core


Thank
You!
OCP
GLOBAL
SUMMIT
October 12-15, 2026
San Jose, California